Documentación de código

Luis Alejandro Gonzalez Juan Sebastian Ulloa David Celeita
Jorge Alberto Castellanos

January 24, 2020

1 Introducción

En el siguiente informe se expone la documentación acerca del software *Ecosystem SafeGuard* desde una perspectiva ingenieril. Se abordará principalmente temas de arquitectura, modularidad y funcionalidad del mismo.

1.1 Propósito del software

El propósito principal del software es generar una detección de ganado, específicamente de origen vacuno, a partir del uso de modelos de visión computacional en Inteligencia Artificial (IA). Para ello, el sistema hace un análisis de imágenes en cualquier formato, mismas imágenes las cuales deben ser tomadas desde vistas aéreas por drones en alturas de un rango entre 50 a 100 metros. El sistema genera reportes de detección, los cuales muestran el conteo de detecciones realizadas en las imágenes, así como las localizaciones precisas en el mapa.

2 Arquitectura del proyecto

El proyecto se maneja en dos capas: una interfaz gráfica que interactúa con el usuario y un script de lógica de detección. Las dos están conectadas mediante una interfaz de función callback como se muestra en la Figura 1.

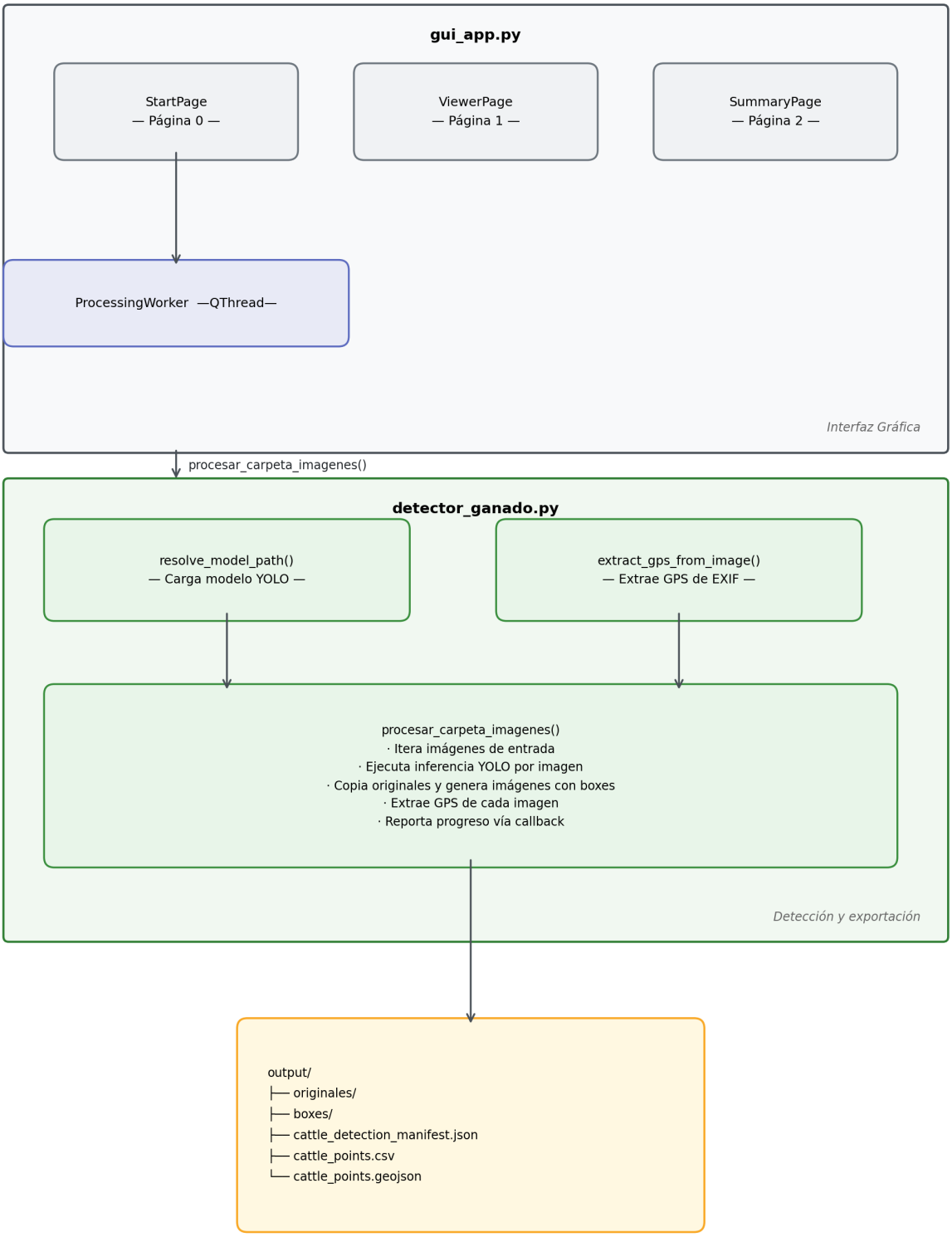

Figure 1: Diagrama arquitectónico del sistema.

Cada capa maneja los componentes listados en la Tabla 1.

Capa	**Responsabilidad**	**Tecnología**
GUI (gui_app.py)	Interfaz de usuario, navegación entre páginas, visualización de imágenes y mapas	PyQt6, QWebEngineView, folium
Detección (detector_ganado.py)	Inferencia YOLO, extracción GPS, generación de archivos de salida	Ultralytics, OpenCV, Pillow
Modelo (models/best.pt)	Modelo entrenado para detección de ganado bovino	Ultralytics YOLO

Table 1: Descripción de las capas del sistema de detección de ganado.

2.1 Máquina de estados del sistema

A continuación, en la Figura 2 se relata la máquina de estados del sistema.

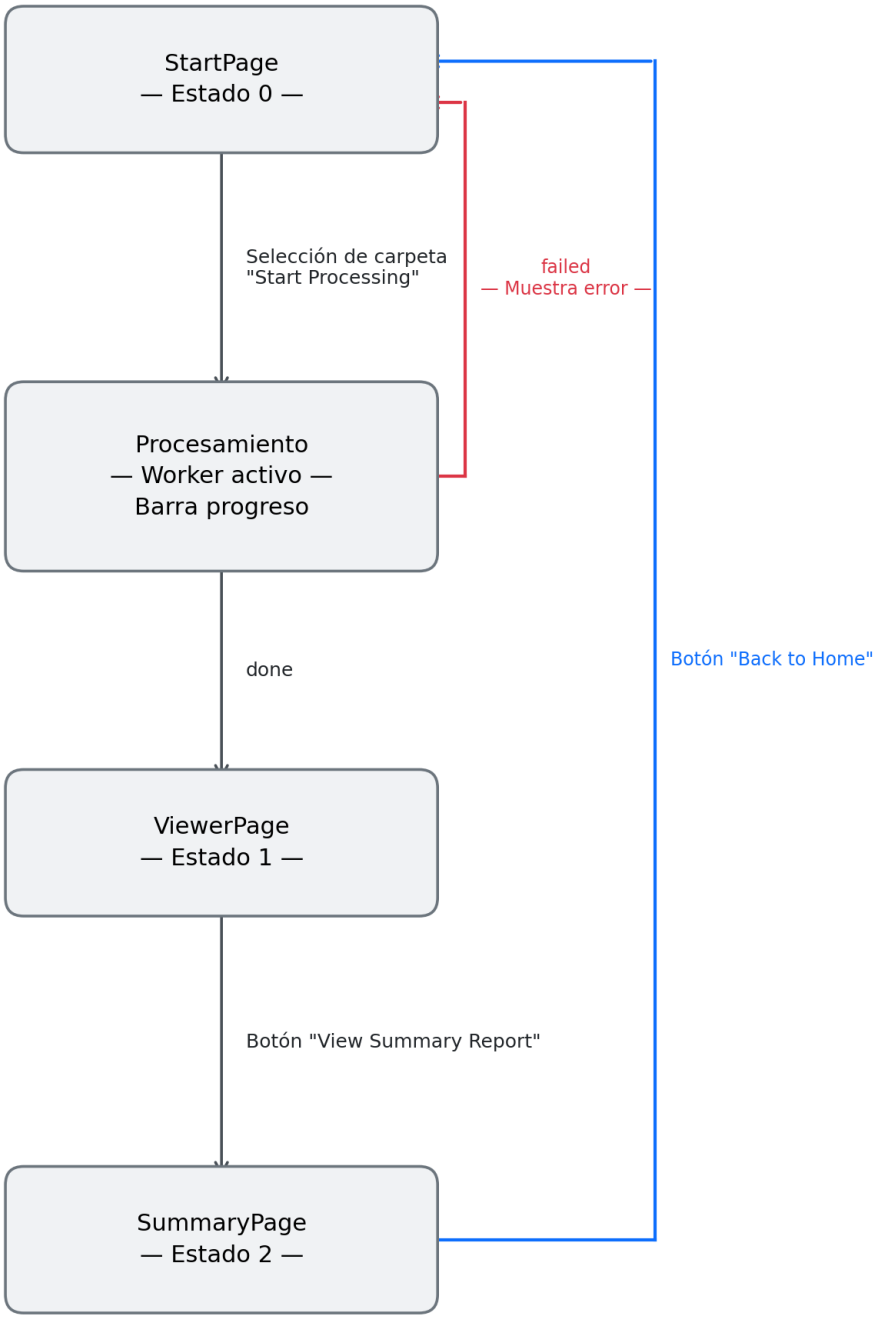

Figure 2: Máquina de estados del sistema.

2.2 Comunicación entre el sistema

Como ya se mencionó, el sistema se maneja en dos capas generales. Las capas se comunican mediante importaciones, específicamente la GUI importa el algoritmo de detección.

La GUI invoca la detección mediante un hilo separado para no bloquear la interfaz:

Listing 1: Invocación por QThread

```
procesar_carpeta_imagenes(
                ruta_carpeta_imagenes=str(self.input_folder),
                ruta_salida=str(self.output_root),
                confianza=0.75,
                iou=0.45,
                img_size=960,
                copiar_originales=True,
                guardar_boxes=True,
                exportar_manifest=True,
                exportar_csv=True,
                clean_output=True,
                progress_cb=_cb,
                hide_conf=True,
                save_conf=False
            )
```

Además de las invocaciones, también hay comunicación entre archivos como se refleja en la Tabla 2.

Archivo generado	**Leído por**	**Propósito**
catlle_detection_manifest.json	`ViewerPage.load_results()`, `SummaryPage.load()`	Lista de imágenes con detecciones, GPS, rutas relativas
boxes/*.jpg	`ViewerPage.show_item()`	Imágenes con bounding boxes dibujados
originales/*.jpg	Exportación al usuario	Imágenes originales de las detecciones
catlle_points.csv	Exportación al usuario	Datos tabulares de detecciones
catlle_points.geojson	Exportación al usuario	Datos geoespaciales estándar

Table 2: Archivos generados por el sistema y su uso.

2.3 Directorios y archivos

A continuación, las Tablas 3 y 4 reflejan los distintos directorios y archivos del proyecto.

Directorio	**Propósito**
`models/`	Contiene el modelo YOLO entrenado (`best.pt`).
`output/`	Directorio de salida generado en tiempo de ejecución. Contiene imágenes procesadas (`originales/`, `boxes/`), archivos de datos (`cattle_detection_manifest.json`, `cattle_points.csv`, `cattle_points.geojson`) y HTMLs de mapas (`_map_current.html`, `_map_summary.html`, `_map_message.html`).
`Logos/`	Logos institucionales mostrados en la barra inferior de la GUI: `EPICS.png`, `Usabana.png`, `Sopo.jpg`.
`Installer/`	Contiene el script Inno Setup para empaquetado.

Table 3: Directorios del proyecto y su propósito.

Archivo	Propósito
`EcosystemSafeguard.spec`	Configuración PyInstaller para generar el ejecutable. Entry point: `gui_app.py`. Incluye `models/best.pt` como dato. Nombre del ejecutable: `EcosystemsSafeguard`.
`CattleDetection.spec`	Configuración PyInstaller alternativa. Recopila PyQt6 y QtWebEngine completos mediante `collect_all()`. Nombre del ejecutable: `CattleDetection`.
`epics-ieee.yaml`	Archivo de entorno Conda (`epics-ieee`) con todas las dependencias del proyecto.
`Installer/EcosystemSafeguard.iss`	Script Inno Setup para generar instalador Windows (`.exe`). Instala en `Program Files`, crea accesos directos en escritorio y menú inicio.

Table 4: Archivos de configuración y empaquetado.

3 Requerimientos de software

Si bien el archivo .yaml del proyecto permite una clonación de las condiciones de software necesarias para el proyecto, se adjunta la lista de requerimientos mínimos del sistema para su funcionamiento en la Tabla 5.

6.1 Sistema Operativo		
Requisito	**Valor verificado**	**Fuente / Nota**
Sistema operativo	Windows x64	Verificado en `EcosystemSafeguard.iss`
Arquitectura	64 bits	`ArchitecturesAllowed=x64`
6.2 Entorno de ejecución		
Componente	**Versión verificada**	**Fuente de verificación**
Python	3.10.19	`epics-ieee.yaml`
Conda	Requerido (entorno `epics-ieee`)	`epics-ieee.yaml`
CUDA Toolkit	12.1	`epics-ieee.yaml` (cuda-runtime=12.1.0)
cuDNN	9	`epics-ieee.yaml` (pytorch=2.5.1-py3.10_cuda12.1_cudnn9_0)
6.3 Paquetes principales		
Paquete	**Versión verificada**	**Propósito en el proyecto**
pytorch	2.5.1 (CUDA 12.1)	Backend de inferencia del modelo
torchvision	0.20.1	Utilidades de visión por computador
ultralytics	8.3.225	Framework YOLOv8 (carga, inferencia, anotación)
opencv-python	4.12.0.88	Escritura de imágenes anotadas (`cv2.imwrite`)
pillow	11.1.0	Lectura de metadatos EXIF/GPS de imágenes
numpy	2.0.1	Operaciones numéricas
PyQt6	(requerido, no listado en yaml — instalado vía pip)	Framework de interfaz gráfica
PyQt6-WebEngine	(requerido, no listado en yaml — instalado vía pip)	Renderizado de mapas HTML en la GUI
folium	(requerido, no listado en yaml — instalado vía pip)	Generación de mapas interactivos Leaflet
6.4 Dependencias secundarias verificadas		
Paquete	**Versión**	**Propósito**
scipy	1.15.3	Cálculos científicos (dependencia de ultralytics)
pandas	2.3.3	Manipulación de datos
matplotlib	3.10.7	Gráficas (dependencia de ultralytics)
seaborn	0.13.2	Visualización estadística
roboflow	1.2.11	Integración con plataforma de datasets
onnxruntime	1.23.2	Runtime ONNX (exportación alternativa de modelos)

Table 5: Requisitos del sistema y dependencias verificadas.